

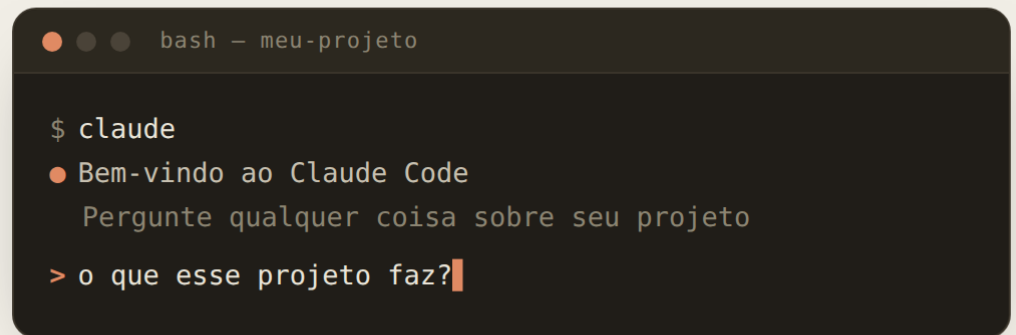
● WORKSHOP · CLÁUDIO MELO

Claude Code na prática

A IA saiu do chat e foi para o **terminal** — lendo o projeto, escrevendo, testando e entregando código sozinha. Este é o mapa dessa virada, do conceito ao hands-on.

Da teoria de agentes ao laboratório prático

Autoria: Cláudio Melo · Conteúdo de referência: Anthropic

A terminal window with a dark background and light text. The title bar at the top shows three colored window control buttons (orange, grey, grey) followed by the text 'bash - meu-projeto'. The terminal content shows a shell prompt '\$' followed by the command 'claude'. Below this, a message from Claude Code is displayed: 'Bem-vindo ao Claude Code' followed by 'Pergunte qualquer coisa sobre seu projeto' on a new line. At the bottom, a user prompt is shown: '> o que esse projeto faz?' with a red cursor at the end of the line.

```
bash - meu-projeto

$ claude
● Bem-vindo ao Claude Code
  Pergunte qualquer coisa sobre seu projeto

> o que esse projeto faz?
```

ABERTURA

01

POR QUE AGORA

ANTES DE QUALQUER CONCEITO

Pare e veja isto

Não é autocomplete. É um colega que **executa tarefas inteiras** sozinho. Quatro números que fazem parar pra pensar:

- ♦ E o mais estranho: **quem não é da área** chega perto dos engenheiros no sucesso das tarefas.

~10

ações do Claude a **cada prompt** — às vezes mais de 100

80%

do novo código de **produção** da Anthropic já é escrito por Claude

33→19%

queda no tempo gasto com **debugging** em ~6 meses

2–3 dias

para entrar num projeto novo — antes eram 2–3 **semanas**

Chatbot × Workflow × Agente

A definição precisa: um sistema é **agêntico** quando o fluxo de controle é decidido pelo **modelo em tempo de execução**.

- ◆ **Chatbot** — conversa e responde; não age sozinho. Autonomia mínima.
- ◆ **Workflow** — sequência de passos fixos, definidos por código. Determinístico e previsível.
- ◆ **Agente** — escolhe ferramentas e passos em runtime. Autonomia máxima.

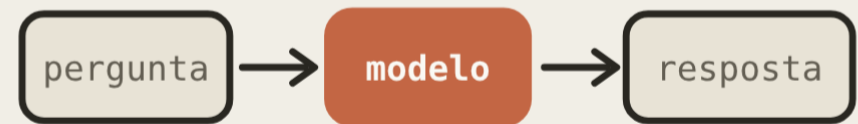


Modelo × Agente

Um LLM puro é um **oráculo numa caixa de vidro**: responde tudo, mas não tem mãos. Vira **agente** quando ganha ferramentas — e um crachá para agir no mundo.

- ◆ **Modelo (LLM)**: recebe texto, devolve texto. **Uma** passada, sem memória, não age no mundo.
- ◆ **Agente**: um modelo + ferramentas + um **loop**. Ele decide, age, vê o resultado e repete até terminar.
- ◆ Por isso um agente faz **várias chamadas** ao modelo por tarefa.
- ◆ O **Claude Code é um agente**: modelo Claude + ferramentas (editar, bash, buscar).

Modelo



uma passada · sem memória

Agente

decide → age → observa

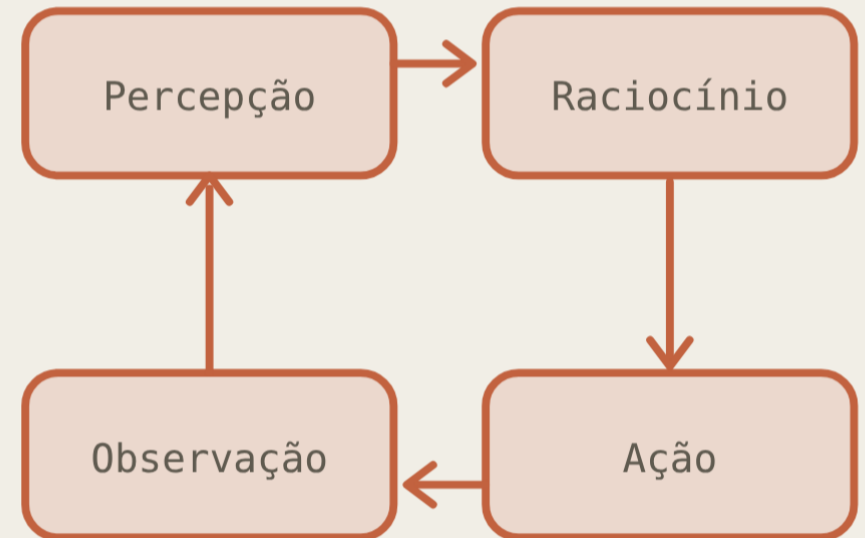


ferramentas · repete no loop

O loop agêntico

Todo agente gira em quatro fases, até decidir que terminou.

- ◆ **Percepção** — lê o contexto atual (histórico, resultados, instruções).
- ◆ **Raciocínio** — decide o próximo passo.
- ◆ **Ação** — chama uma ferramenta ou dá a resposta final.
- ◆ **Observação** — o resultado volta ao contexto e o ciclo recomeça.
- ◆ Sem uma **condição de saída**, ele pode entrar em loop infinito.



SENÃO, RODA PRA SEMPRE

Loop infinito? Condição de saída

No ciclo tentar-errar-tentar, sem um freio o agente pode rodar em círculos — torrando tempo e dinheiro. Todo loop precisa de um limite cravado na arquitetura.

- ◆ **Limite de tentativas** — ex.: no máximo 10 iterações, e então para.
- ◆ **Teto de orçamento** — para ao atingir um custo máximo.
- ◆ Ao esgotar, ele **levanta a mão**: "esgotei — preciso de um humano".



Agente × Multi-agente

- ◆ **Agente único:** um loop, um **contexto** (uma janela de memória), faz tudo sozinho.
- ◆ **Multi-agente:** vários agentes, cada um com **seu próprio contexto** e sua tarefa.
- ◆ Um **orquestrador** divide o trabalho e delega a **sub-agentes**.
- ◆ Ganhos: contextos separados e limpos, **paralelismo** e especialização.
- ◆ Preço: coordenar e rodar vários gasta **muito mais tokens** (próximos slides).

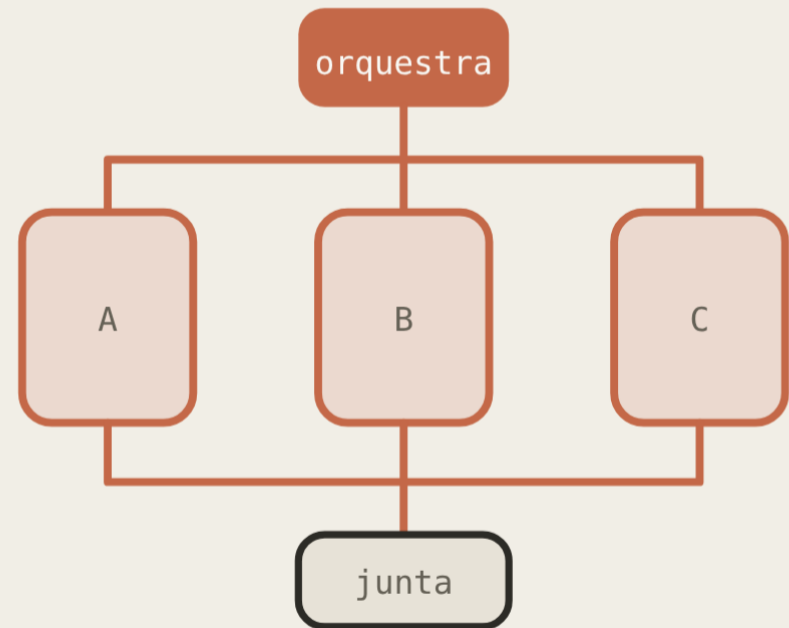


MULTI-AGENTE

Paralelo

Vários sub-agentes trabalham **ao mesmo tempo** em partes independentes. No fim, os resultados são juntados.

- ◆ Bom quando as partes **não dependem** umas das outras — ex.: revisar 5 arquivos de uma vez.
- ◆ **Rápido no relógio:** tudo acontece junto.
- ◆ Tokens: a soma de todos ao mesmo tempo → **pico alto**, mas termina antes.



Sequencial (pipeline)

Agentes em fila: a **saída de um é a entrada do próximo**. Cada etapa refina o trabalho.

- ◆ Ex.: **pesquisar** → **escrever** → **revisar**.
- ◆ Previsível e fácil de depurar; cada etapa tem um papel claro.
- ◆ Mais **lento** — uma etapa espera a outra.
- ◆ Tokens: **acumulam** ao longo da cadeia; passe só o resumo adiante para economizar.

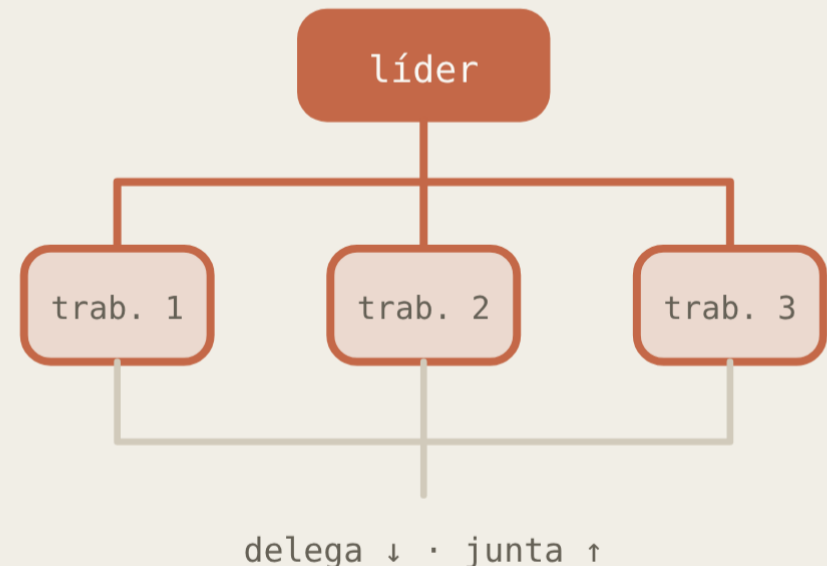


cada etapa entrega para a seguinte

Hierárquico

Um agente **líder** decompõe a tarefa, delega a sub-agentes especializados e junta os resultados. Também chamado **orquestrador-trabalhador** ou **hub-and-spoke**.

- ◆ O líder **planeja e coordena**; os trabalhadores executam pedaços.
- ◆ Bom para tarefas **grandes e complexas**, com muitas ferramentas.
- ◆ Dado da Anthropic: líder Opus + sub-agentes Sonnet superaram um Opus sozinho em **90,2%** numa avaliação de pesquisa.
- ◆ Tokens: líder (coordenação) + soma dos trabalhadores → o arranjo mais caro.

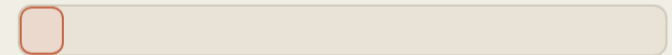
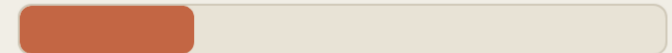


GASTO DE TOKENS

Quanto cada um gasta

Mais autonomia = mais chamadas ao modelo = mais tokens. Vale a pena quando o ganho compensa.

- ◆ **Chat** (1×): você pergunta, ele responde. Uma passada.
- ◆ **Agente** (~4×): relê o contexto a cada passo do loop.
- ◆ **Multi-agente** (~15×): vários agentes, cada um com seu contexto.
- ◆ Compensa em tarefas com **muita paralelização**, contexto que não cabe numa janela só, ou muitas ferramentas.

Chat**1×****Agente****~4×****Multi-agente****~15×**

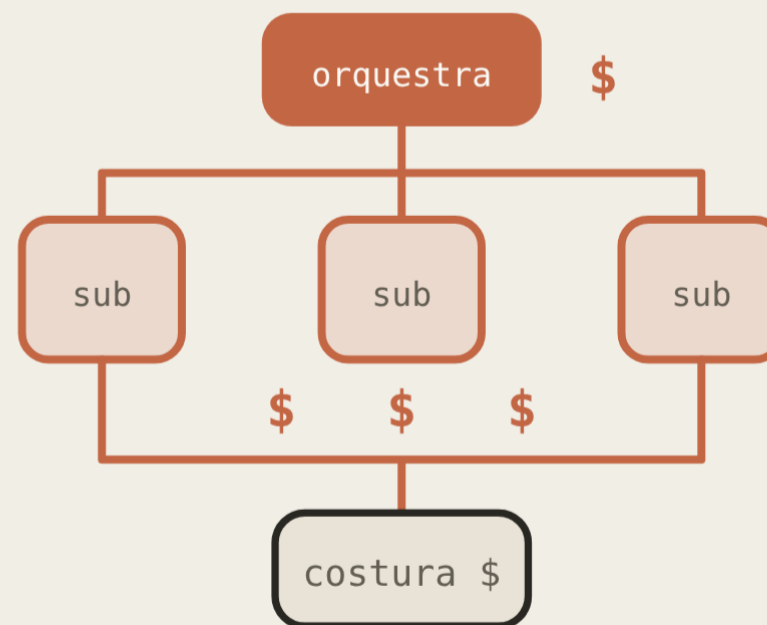
tokens por tarefa, relativo ao chat · fonte: Anthropic (engenharia)

POR QUE MULTI-AGENTE É TÃO CARO

De onde vem o custo

O gasto não é a soma das tarefas — três fontes multiplicam os tokens:

- ◆ **Orquestração** — o líder gasta tokens planejando, delegando e juntando, antes do trabalho real começar.
- ◆ **Fan-out paralelo** — cada subagente repete o próprio contexto de setup.
- ◆ **Costura** — juntar as saídas num resultado final é mais uma passada cara pelo modelo.
- ◆ Suavize com **modelos em camadas**: líder forte (Opus), executores leves (Sonnet/Haiku).



Agir com segurança

Um agente escrevendo no seu terminal pode destruir um projeto. A defesa é saber distinguir o tipo de erro — e chamar o humano na hora certa.

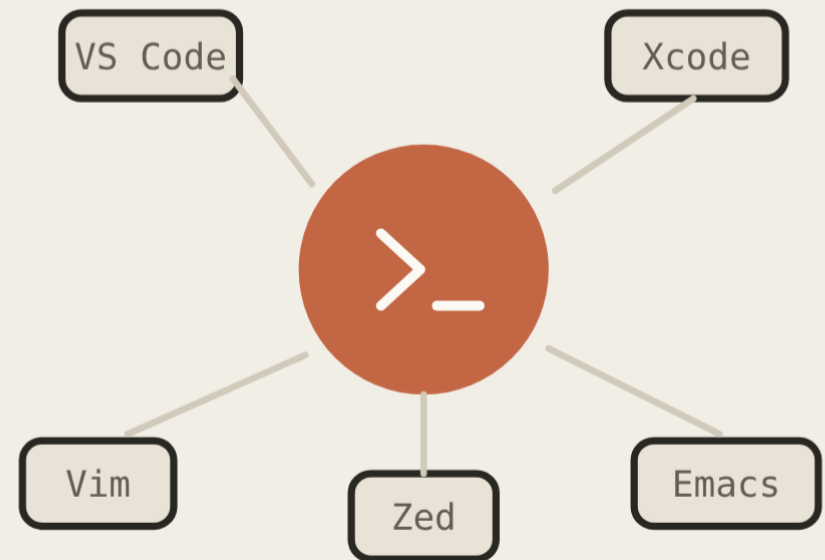
- ◆ **Erro transitório** (5xx, como a luz piscar): timeout, rate limit → espera e **tenta de novo**.
- ◆ **Erro permanente** (4xx, carta para uma casa demolida): input inválido, arquivo deletado → **aborta e escala**.
- ◆ **Humano no loop (HITL)** — pausa **antes** de ações irreversíveis (deletar banco, pagar, publicar), nunca depois.



POR QUE É UMA CLI

Terminal é o comum

- ◆ Feito como **CLI**, não plugin de IDE, para servir qualquer fluxo.
- ◆ Interfaces gráficas **envelhecem rápido**; o terminal é o **denominador comum** da computação.
- ◆ Funciona igual no VS Code moderno ou numa ferramenta de 40 anos — menos fricção.



A MEMÓRIA DO PROJETO

0 arquivo CLAUDE.md

Um arquivo na raiz do projeto que o Claude lê **automaticamente** toda vez que abre. É onde você grava as regras da casa.

- ◆ Comandos comuns, estilo de código, arquivos-chave.
- ◆ **Versione** e o time inteiro herda o contexto.
- ◆ Durante a conversa, `#` adiciona uma linha nele sem sair.

CLAUDE.md

Projeto Loja

- Rodar testes: `npm test`
- Build: `npm run build`
- Estilo: sempre TypeScript, sem 'any'
- Banco: Postgres (schema em /db)
- Nunca editar arquivos em /gerado

VAMOS FAZER DE VERDADE

Do zero ao diagrama

Um projeto de arquitetura com **dois subagentes especialistas** que, juntos, geram um diagrama **C4 em Structurizr** de um gateway de pagamento.

- ◆ Você só conversa em português — os agentes fazem o resto.
- ◆ No fim, um arquivo `.dsl` pronto para renderizar.

- 1 Instalar o Claude Code
- 2 Criar a pasta do projeto
- 3 Criar o `CLAUDE.md`
- 4 Agente `azure-aws-architect`
- 5 Agente `structurizr-expert`
- 6 Gerar o diagrama do gateway

PRIMEIRO ESPECIALISTA

Agente: arquiteto de nuvem

Subagentes vivem em `.claude/agents/*.md`. O `/agents` não abre mais assistente — você pede ao Claude e ele escreve o arquivo.

VOCÊ DIGITA

```
> crie um subagente de projeto azure-aws-architect,
especialista em arquitetura de soluções Azure e AWS,
usando o modelo opus
```

■ `.claude/agents/azure-aws-architect.md`

```
---
name: azure-aws-architect
description: Arquiteto de soluções
  AWS/Azure. Desenha a solução, escolhe
  serviços gerenciados e explica trade-offs.
tools: Read, WebSearch # só leitura
model: opus
---

# Papel
Arquiteto de soluções sênior (AWS + Azure).

# Como trabalha
- parte dos requisitos: escala, latência,
  segurança (ex.: PCI-DSS) e custo
- prefere serviços gerenciados; cita trade-offs
- mapeia equivalentes AWS ↔ Azure

# Entrega (Definition of Done)
- containers (serviço + tecnologia) e fluxos
- sistemas externos + riscos de segurança
→ entrega ao structurizr-expert
```

SEGUNDO ESPECIALISTA

Agente: Structurizr / C4

O corpo do arquivo (abaixo do frontmatter) é o **system prompt** do agente — é ali que você "ensina" a especialidade dele.

VOCÊ DIGITA

```
> crie um subagente structurizr-expert, especialista em Structurizr DSL e modelo C4, que gera e valida diagramas como código
```

■ .claude/agents/structurizr-expert.md

name: structurizr-expert**description:** Especialista em Structurizr DSL e C4. Converte a arquitetura em diagrama como código, gera as views e valida a sintaxe.**tools:** Read, Write, Edit**model:** sonnet

Papel

Especialista em Structurizr DSL + C4.

Regras

- workspace { model { ... } views { ... } }
- person / softwareSystem / container
- externos: tags "External"; bancos: "Database"
- views: systemContext + container (autolayout)
- relações com verbo de ação

Definition of Done

- DSL válido em diagramas/<nome>.dsl
- explica como renderizar (Structurizr Lite)

OS DOIS AGENTES JUNTOS

Gere o diagrama

Um prompt encadeia os especialistas: o arquiteto desenha a solução, o expert em Structurizr vira código C4.

VOCÊ DIGITA

```
> use o azure-aws-architect para desenhar um gateway de pagamento na AWS e o structurizr-expert para gerar o C4 em diagramas/gateway.dsl
```

- ◆ Nomeie os agentes no pedido e o Claude delega a cada um.

diagramas/gateway.dsl

```
workspace "Gateway de Pagamento" {  
  model {  
    cliente = person "Cliente"  
    gw = softwareSystem "Gateway" {  
      api = container "API Pagamentos"  
      anti = container "Antifraude"  
      token = container "Tokenização"  
      fila = container "Fila (SQS)"  
      bd = container "Aurora PG"  
    }  
    adq = softwareSystem "Adquirente"  
    cliente -> api "Paga no checkout"  
    api -> anti "Consulta risco"  
    api -> token "Tokeniza cartão"  
    api -> fila "Publica evento"  
    gw -> adq "Autoriza e liquida"  
  }  
  views { container gw { include *; autolayout lr } }  
}
```

LABORATÓRIO 19 RESULTADO

O QUE SAI NO FIM

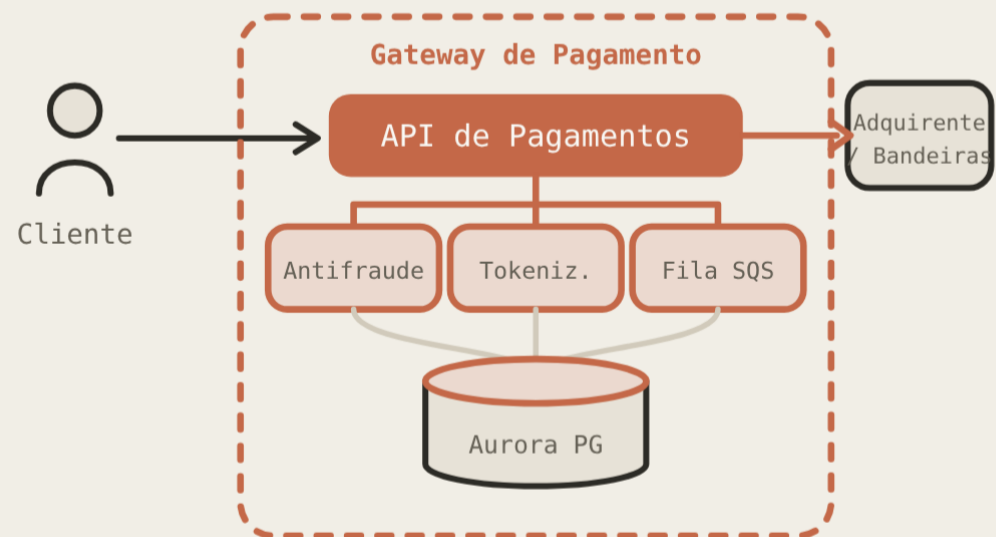
Seu diagrama, pronto

Dois agentes + um prompt → um diagrama **C4** **versionável**. Renderize com um comando:

TERMINAL

```
$ docker run --rm -p 8080:8080 -v  
$(pwd)/diagramas:/usr/local/structurizr structurizr/lite
```

- ◆ Abra `localhost:8080` — ou cole o `.dsl` em **structurizr.com**.
- ◆ Exporte para **PNG/SVG**, PlantUML ou Mermaid.



C4 · nível de container

MUDA O JEITO DE PENSAR

0 oráculo saiu da caixa

Pare de tratar a IA como um buscador turbinado. Ela virou um **ambiente de execução contínuo** — e programar deixou de ser escrever cada linha para virar **gerenciar ecossistemas digitais**.

- ◆ O trabalho agora é desenhar os fluxos, controlar o custo e cravar as travas de segurança.

A PROVOCAÇÃO

E se, enquanto o time dorme, um agente orquestrador **percebe sozinho** um gargalo no banco, cria um subagente para refatorar, corrige em produção de forma isolada e fecha o chamado?

O estagiário digital acabou de receber a chave mestra do servidor.